

Kubernetes Error 1: CrashLoopBackOff

What is CrashLoopBackOff?

- A CrashLoopBackOff error occurs when a container starts , crashes , and Kubernetes repeatedly tries to restart it.
- After several failed attempts, Kubernetes delays the restart , resulting in the CrashLoopBackOff status.
- 🚀 Common causes include *applicationcrashes* , *configuration errors, missingSecrets* , *dependency failures*, *resourcelimitsprobefailures* , and *storage or permissionissues* .

Kubernetes CrashLoopBackOff — Common Reasons & Solutions

 Reason	 Description	 Solution	 Interview Tip
 Application Crash	Application throws an exception and exits immediately	Check logs using <code>kubectll logs <pod-name></code> and fix the application	Most common cause
 Incorrect Startup Command	Invalid command or args	Verify command and args in the Deployment	Check the container entry point
 Missing Environment Variables	Required environment variables are missing	Add missing variables under <code>env</code>	Verify ConfigMaps and Secrets
 Configuration Errors	Invalid ConfigMap or configuration files	Validate ConfigMaps and mounted files	Check mounted configuration
 Missing Secrets	Secret is missing or incorrect	Create/update the Secret and verify references	Confirm secret names and keys
 Database Connection Failure	Application cannot connect to the database	Verify hostname, credentials, Service, and network	Ensure the database is reachable

 Reason	 Description	 Solution	 Interview Tip
 Image Issues	Wrong image or outdated version	Verify image name/tag and redeploy	Confirm the correct image is available
 Application Dependency Failure	Redis, Kafka, external API, etc. unavailable	Ensure dependent services are running	Verify connectivity to dependencies
 OOMKilled	Container exceeds its memory limit	Increase memory limits or optimize the application	Check Pod status for <code>OOMKilled</code>
 CPU Resource Limits	Insufficient CPU resources	Increase CPU requests and limits	Monitor CPU usage
 Liveness Probe Failure	Kubernetes restarts the container because it appears unhealthy	Verify probe path, port, and timings	Ensure the health endpoint is correct
 Startup Probe Failure	Slow-starting application fails during initialization	Increase <code>failureThreshold</code> or <code>initialDelaySeconds</code>	Recommended for Java/Spring Boot apps
 Read-only File System	Application writes to a read-only path	Mount a writable volume or change the write location	Common with hardened containers
 Port Configuration Error	Application listens on a different port	Match the container port with the application port	Verify <code>containerPort</code>
 Permission Issues	Application lacks file or directory permissions	Fix file permissions, <code>securityContext</code> , or user	Common when running as a non-root user
 Volume Mount Failure	PVC, ConfigMap, or Secret isn't mounted correctly	Verify PVC status and volume mounts	Check <code>kubectl describe pod</code>
 Node-Level Issues	Node resource pressure or	Check node health using <code>kubectl describe node <node-name></code>	Review node events and conditions

 Reason	 Description	 Solution	 Interview Tip
	kubelet problems		

Troubleshooting Steps

Check Pod Status

```
kubectl get pods
---
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-7f9c6d9b8c-abcde	0/1	CrashLoopBackOff	5	10m

Troubleshooting Flow



Frequently Used Commands

kubectl get pods	# Check pod status
kubectl logs <pod-name>	# View Container Logs
kubectl logs <pod-name> --previous	# View previous logs
kubectl describe pod <pod-name>	# Describe pod
kubectl edit deployment <deployment-name>	# Edit deployment
kubectl rollout restart deployment <deployment-name>	# Restart deployment
kubectl get pods -w	# Watch pod status

Interview Answer

Q: How do you troubleshoot a CrashLoopBackOff error ?

- Check the Pod events using ***kubectl describe pod***.
- Review the application logs with ***kubectl logs*** (and ***--previous*** if needed).
- Verify the ***startup command, environment variables, ConfigMaps***, and ***Secrets***.
- Check ***database*** and ***external service connectivity***.
- Inspect ***CPU*** and ***memory*** usage, especially for ***OOMKilled***.
- Validate ***liveness*** and ***startup probe configuration***.
- Verify ***volume mounts, file permissions***, and ***node health***.